AES

GUIA RÁPIDO · V0.1.0

Agent Engineering System
Engenharia consistente para agentes de código.

O que é o AES, como ele funciona, como instalar em um projeto e como usar o ciclo de inovação contínua com segurança.

Codex | Claude Code | Google Antigravity | GitHub Copilot

Versão
0.1.0

Data
27 de outubro de 2026

Requisitos
Python 3.10+ · sem dependências

Licença
MIT

Repositório
github.com/Httromrj/AES-agent-engineering-system

01 O que é o AES

O **Agent Engineering System** é um conjunto de padrões de engenharia, *skills* e ferramentas que faz agentes de código (Codex, Claude Code, Antigravity e Copilot) trabalharem com o mesmo nível de rigor em qualquer projeto. Você escreve as regras **uma vez**, em um único lugar, e o instalador gera os arquivos que cada agente lê nativamente.

O problema que ele resolve: cada ferramenta procura instruções em um arquivo diferente (`AGENTS.md`, `CLAUDE.md`, `.github/copilot-instructions.md`…). Manter cópias manuais gera divergência; o agente acaba pulando investigação, testes ou cuidados de segurança.

Princípios

Fonte única
Padrões vivem em `core/` e `skills/`. Adaptadores são gerados, nunca editados à mão.

Revelação progressiva
Regras curtas sempre ativas; skills detalhadas carregadas só quando a tarefa pede.

Evidência acima de alegações
O agente só relata testes e verificações que realmente executou.

Instalação segura
Preserva seu CLAUDE.md/AGENTS.md: adiciona um bloco próprio; atualiza com backup e rollback.

Melhoria revisada
Melhorias viram propostas com evidência; nada se reescreve sozinho.

Sem dependências
Somente biblioteca padrão do Python; fácil de auditar e de rodar em qualquer lugar.

02 Como funciona

AUTORIA
core/ + skills/
Padrões e procedimentos canônicos

→

INSTALAÇÃO
install.py
Copia o núcleo e gera adaptadores

→

PROJETO
.agent-engineering/
Cópia versionada + manifest com hashes

→

AGENTES
Arquivos nativos
AGENTS.md, CLAUDE.md, skills…

O que vem no repositório

Pasta	Conteúdo
`core/`	Carta de engenharia, fluxo de trabalho, verificação e riscos, qualidade de produto/UX, protocolo de melhoria contínua e ADR-0001.
`skills/`	Cinco skills: *codebase-investigation*, *architecture-review*, *implementation-quality*, *product-engineering* e *continuous-improvement*.
`scripts/`	`install.py` (instala/atualiza), `innovation.py` (varredura de sensibilidade, pesquisa e registro), `gateway.py` (interface opcional para modelos), `dashboard.py` (painel web), `validate.py` e `vps_preflight.py`.
`research/`	Fontes RSS/Atom curadas (`sources.json`) e exemplo de configuração do gateway.
`evals/`	Tarefas de referência e rubrica de 0 a 14 pontos para comparar o desempenho dos agentes antes e depois de mudar o AES.
`deployment/`	Dockerfile para Coolify, modelo systemd e instruções de operação em VPS.

O fluxo que o agente passa a seguir

1 **Triagem** — Resultado, critérios, risco › 2 **Investigar** — Código, testes, contratos › 3 **Desenhar** — Alternativas e falhas › 4 **Implementar** — Mudança mínima e completa › 5 **Verificar** — Testes reais, sem suposição › 6 **Revisar** — O que pode quebrar? › 7 **Melhorar** — Propor com evidência

A profundidade acompanha o risco. Mudanças **baixas** (texto, estilo) pedem pouco; **normais** cobrem um fluxo delimitado; **altas** envolvem autenticação, pagamentos, dados pessoais, migrações ou integrações; **sensíveis à produção** exigem autorização explícita.

03 Como aplicar em um projeto

1 · Validar o AES

```
# na pasta do repositório AES
python scripts/validate.py
python -m unittest discover -s scripts -p "test_*.py"
```

2 · Instalar no projeto

```
# escolha os agentes que o time usa
python scripts/install.py C:\caminho\do\projeto --agents codex,claude,antigravity,copilot
```

Agente	Entrada gerada	Skills
Codex	`AGENTS.md`	`.agents/skills/`
Claude Code	`CLAUDE.md` (importa o `AGENTS.md`)	`.claude/skills/`
Google Antigravity	`AGENTS.md` + `.agents/rules/aes.md`	`.agents/skills/`
GitHub Copilot	`.github/copilot-instructions.md`	`.github/skills/`

Projeto já tem `CLAUDE.md` ou `AGENTS.md`? Sem problema. O AES insere um bloco delimitado (`<!-- AES:BEGIN -->` … `<!-- AES:END -->`) no topo do arquivo e mantém todo o seu conteúdo abaixo, inclusive frontmatter e quebras de linha. No `--update`, só o bloco é trocado.

3 · Revisar e confirmar

- Revise o diff (`git diff`) e faça commit de `.agent-engineering/` e dos adaptadores gerados.
- Abra uma sessão nova do agente e pergunte quais instruções ele carregou, para confirmar a descoberta.
- Escreva regras específicas do projeto (comandos de build, convenções) **fora** do bloco do AES, em qualquer lugar do arquivo. Alterações dentro do bloco fazem o `--update` parar e avisar.

4 · Atualizar quando o AES evoluir

```
python scripts/install.py C:\caminho\do\projeto --update
```

O atualizador compara cada arquivo (e cada bloco do AES) com o hash do manifest. Se você editou algo gerenciado pelo AES, ele **recusa** a atualização e aponta o arquivo. A atualização é preparada em área temporária e aplicada com backup; se uma escrita falha, tudo volta ao estado anterior.

Regra de ouro: altere padrões no repositório do AES (`core/`, `skills/`), não nas cópias dentro dos projetos. Depois rode `--update` em cada projeto.

04 Ciclo de inovação contínua

O AES acompanha novidades de fontes oficiais (OpenAI, GitHub, Claude Code, Codex, VS Code, Google, Python, W3C) e as transforma em uma fila de leitura. Cada item é uma **hipótese** até alguém verificar a fonte primária.

COLETAR	→	TRIAR	→	VERIFICAR	→	PROPOR
research		**Painel**		**Fonte primária**		**record proposal**
Feeds HTTPS, limitados e sem modelo		Interessante ou descartar		Comparar com alternativas		Critério de sucesso e decisão

```
# coletar novidades (também roda semanalmente no Coolify)
python scripts/innovation.py research --per-source 10

# registrar uma proposta verificada (JSON com evidência, alternativas, critério de sucesso)
python scripts/innovation.py record proposal proposta.json

# antes de enviar arquivos a um modelo externo, publicar ou compartilhar
python scripts/innovation.py scan caminho/ --action external_model
```

Da novidade para os seus projetos

Em *Descobertas*, um admin clica em **Implantar nos projetos**. A IA escolhida em *IA e GitHub* (Claude, Gemini, DeepSeek, Grok ou OpenAI) lê a fonte e escreve uma **skill**. Você confere a prévia e confirma; o AES faz **commit direto na main** de cada repositório escolhido, em `.claude/skills/` (e `.agents/`, `.github/` quando detectados). Em *Implantações*, cada commit tem link e botão **Reverter**. Variáveis no Coolify: a chave do provedor (ex.: `GEMINI_API_KEY`) e `AES_GITHUB_TOKEN`.

Portão de sensibilidade: qualquer achado (chave, token, e-mail, string de conexão, marcador de confidencialidade) ou falha de leitura **bloqueia** a ação. O relatório mostra só caminho e tipo de detector, nunca o valor. Um resultado limpo não prova que o arquivo é seguro.

05 Painel de inovação e acesso

O painel web (`scripts/dashboard.py`) roda no Coolify com HTTPS e autenticação. Ele mostra o estado das pesquisas, permite triar descobertas e implantar skills nos projetos cadastrados.

Tela	Para que serve
Visão geral	Fila de triagem, itens novos, fontes respondendo, última execução e gráfico de atividade.
Descobertas	Caixa de entrada com busca, filtros por fonte e estado, e painel de detalhes.
Propostas	Decisão mais recente de cada proposta (proposta, aceita, rejeitada, adiada).
Fontes	Status de cada feed, tópicos e quantidade de itens.
Execuções	Histórico de ciclos com duração, itens lidos, novos e erros.
Implantações	Skills geradas, prévia, commits por projeto e botão Reverter.
Projetos · IA e GitHub	Repositórios que recebem skills; escolha do provedor de IA e status do token.
Digest	Resumo em Markdown do dia, com uma seção por execução.

Atalhos na caixa de entrada

`/` buscar · `J` / `K` navegar · `I` interessante · `D` descartar · `U` voltar à fila · `O` abrir a fonte · `Esc` fechar

Usuário e senha

O painel tem tela de login, vários usuários e dois papéis: **administrador** (gerencia usuários) e **membro** (usa o painel, faz triagem e roda pesquisas). As senhas são guardadas com hash e, após 5 tentativas erradas, a conta fica bloqueada por 5 minutos. A **conta de recuperação** vem das variáveis `AES_DASHBOARD_USER` e `AES_DASHBOARD_PASSWORD` no Coolify e sempre funciona como administrador.

Situação	O que fazer
Criar usuário	*Usuários → Novo usuário*. O painel mostra uma senha temporária (uma única vez); a pessoa cria a própria senha no primeiro acesso.
Trocar a senha	*Minha conta → Alterar senha*. As outras sessões da conta são encerradas.
Esqueci a senha	Um administrador clica em *Redefinir senha* e entrega a senha temporária. O link "Esqueci minha senha" no login explica o caminho.
Admin sem acesso	Entre com a conta de recuperação ou rode `python scripts/dashboard.py users reset NOME` no terminal do container.
Sair	Botão ao lado do seu nome, no rodapé do menu lateral.

06 Boas práticas e limites

Faça

- Meça com `evals/benchmark-tasks.md` antes e depois de mudar padrões.
- Prefira um teste ou checagem automática a uma instrução vaga.
- Registre decisões aceitas, rejeitadas e adiadas, com justificativa.
- Faça backup do volume de estado do painel fora da VPS.

O AES não faz

- Não garante que o agente siga as instruções; revisão humana continua necessária.
- Não verifica alegações de produtos nem roda experimentos sozinho.
- Não grava código da aplicação: nos projetos, só commita skills (`SKILL.md`) aprovadas por um admin.
- Não chama modelos externos na pesquisa; o gateway vem sem provedores.